

Below is a documentation-focused background summary. It describes each technology independently and avoids comparative judgments or recommendations.

1. Kubernetes

Core architecture

Kubernetes is an open-source container orchestration platform organized around a **control plane** and **worker nodes**.

The control plane includes:

- **kube-apiserver** — exposes the Kubernetes API and serves as the primary control-plane interface.
- **etcd** — consistent, highly available key-value store containing cluster state.
- **kube-scheduler** — assigns unscheduled Pods to appropriate nodes.
- **kube-controller-manager** — runs controllers that continuously reconcile desired and actual state.
- **cloud-controller-manager** — optional component integrating Kubernetes with cloud-provider infrastructure.

Worker nodes run components including:

- **kubelet** — ensures the containers specified by Pods are running and healthy.
- **kube-proxy** — provides network proxy functionality for Services, where applicable.
- A **container runtime** and networking components implementing the required interfaces.

Kubernetes supports both self-managed clusters and managed Kubernetes services, with control-plane components capable of being distributed across multiple machines for high availability. ([Kubernetes](#))

Scalability limits

For Kubernetes **v1.37**, the official large-cluster guidance specifies a design target of:

Specification	Documented value
Maximum nodes per cluster	5,000
Maximum Pods per cluster	150,000
Maximum containers per cluster	300,000
Recommended maximum Pods per node	110

These figures describe the Kubernetes project's documented scalability target rather than an absolute physical limit applicable to every deployment. Cloud-provider quotas, networking, storage, API-server load and workload characteristics can impose additional constraints. ([Kubernetes](#))

The API server can itself be horizontally scaled by running multiple instances behind load balancing. Kubernetes also supports adding/removing worker nodes to scale cluster capacity. ([Kubernetes](#))

Setup complexity

A self-managed Kubernetes cluster requires substantially more infrastructure components than simply installing a container engine.

For a kubeadm-based installation, the official prerequisites include:

- Linux hosts
- At least **2 GiB RAM per machine**
- At least **2 CPUs for a control-plane machine**
- Full network connectivity between machines
- Required network ports
- A container runtime
- `kubeadm`
- `kubelet`
- `kubectl`
- A Pod networking implementation

The Kubernetes documentation explicitly describes production-like kubeadm setup as an advanced task requiring multiple machines and careful configuration. Production high availability additionally involves distributing control-plane components and managing etcd appropriately. ([Kubernetes](#))

Managed Kubernetes services can abstract away some of this control-plane administration, but the workload cluster still requires configuration of networking, storage, identity, security, observability and application resources. ([Kubernetes](#))

Cost structure

Kubernetes itself is open-source software; there is no Kubernetes software license fee for running the upstream project.

For a **self-managed** installation, costs instead arise from the infrastructure and operational components required to run it, such as:

- Control-plane servers
- Worker compute
- Persistent storage
- Networking and load balancers

- IP addresses and data transfer
- Container registry
- Monitoring/logging
- Backup infrastructure
- Operations and engineering labor

For a **managed Kubernetes service**, the cloud provider may additionally charge for the managed control plane and associated cloud resources. The exact structure depends on the provider.

Industry statistics

The CNCF's 2025 Annual Cloud Native Survey, published in January 2026, surveyed **628 respondents**. It reported that **82% of container users were running Kubernetes in production**, while **98% of surveyed organizations reported adopting cloud-native techniques**. The survey also reported Kubernetes use for generative-AI workloads among **66% of organizations hosting generative-AI models**. These are survey statistics, not measurements of the entire global IT market. ([CNCf](#))

2. Docker Swarm

Core architecture

Docker Swarm, specifically **Swarm mode**, is built directly into Docker Engine. It does not require a separate orchestration product. ([Docker Documentation](#))

A Swarm consists of Docker Engine nodes with two primary roles:

- **Manager nodes**
 - Maintain cluster state
 - Schedule services
 - Provide the Swarm API
 - Maintain state using **Raft consensus**
- **Worker nodes**
 - Execute scheduled tasks/containers

Managers can also execute workloads unless configured otherwise. ([Docker Documentation](#))

The basic deployment unit is a **service**. A service specifies the desired state, including:

- Container image
- Replica count
- Ports
- CPU/memory constraints
- Networking

- Placement requirements
- Update behavior

The Swarm manager continuously reconciles actual state against the declared desired state. ([Docker Documentation](#))

Built-in capabilities include:

- Overlay networking
- Service discovery through embedded DNS
- Internal load balancing
- Mutual TLS between nodes
- Rolling updates
- Service replication and reconciliation ([Docker Documentation](#))

Scalability limits

Docker's current documentation does **not specify a hard maximum number of worker nodes or containers equivalent to Kubernetes' published 5,000-node/150,000-Pod target.**

Documented architectural constraints include:

- A Swarm requires at least one manager.
- Worker nodes increase workload capacity.
- Manager nodes participate in Raft consensus.
- Docker recommends **no more than seven manager nodes** for a Swarm.
- Adding managers increases fault tolerance but does not increase scalability/performance; additional managers can reduce write performance because more nodes must acknowledge state changes. ([Docker Documentation](#))

For high availability, Docker documents the fault tolerance of odd-numbered manager configurations:

Managers	Simultaneous manager failures tolerated
1	0
3	1
5	2
7	3

The seven-manager figure is a Docker recommendation rather than a protocol-level maximum. ([Docker Documentation](#))

Setup complexity

Swarm is integrated into Docker Engine. Enabling it involves initializing a Docker Engine as a manager and joining additional Docker Engines as workers or managers.

Docker's introductory Swarm tutorial uses:

- **Three networked Linux hosts**
- Docker Engine installed on each
- Network connectivity between the hosts
- A manager-node IP address
- Required protocols/ports

The basic workflow is therefore centered around Docker Engine and Docker CLI commands rather than installing a separate orchestration control plane. ([Docker Documentation](#))

Cost structure

Swarm mode is part of Docker Engine. Docker Engine is an open-source technology under the **Apache License 2.0**. ([Docker Documentation](#))

The infrastructure cost therefore depends primarily on the environment used to run the Docker Engines:

- Physical or virtual machines
- Cloud compute
- Storage
- Network traffic
- Load balancers
- Container registry
- Monitoring/logging
- Backup and operations

Docker also has commercial products and subscription requirements in certain Docker Desktop enterprise-use scenarios; those licensing rules are distinct from the underlying open-source Docker Engine/Swarm technology. ([Docker Documentation](#))

3. AWS Fargate

Core architecture

AWS Fargate is a **serverless compute engine for containers**, rather than a standalone container orchestrator.

It provides compute capacity for:

- **Amazon ECS tasks**
- **Amazon EKS Pods**

AWS manages the underlying compute infrastructure while users specify container workloads and their required resources. Fargate therefore removes the requirement for the customer to provision and manage the underlying EC2 instances for those workloads. ([Amazon Web Services, Inc.](#))

The resource model is task/Pod oriented. CPU and memory are requested at the task level for Fargate workloads.

Resource/scalability specifications

Current ECS/Fargate task configurations include:

vCPU	Supported memory range
0.25	0.5–2 GB
0.5	1–4 GB
1	2–8 GB
2	4–16 GB
4	8–30 GB
8	16–60 GB
16	32–120 GB
32	60, 120, or 244 GB

Some configurations have OS/platform restrictions; for example, the higher CPU configurations are documented for Linux, while Windows support has a narrower configuration range. ([AWS Documentation](#))

Fargate scaling is also subject to AWS service quotas. Current ECS documentation, for example, lists regional quotas for:

- Fargate On-Demand concurrent vCPU resources
- On-Demand burst task launches
- On-Demand sustained task-launch rate
- Tasks launched per service per minute

These quotas can be adjustable, and AWS notes that new accounts can initially have lower quotas. ([AWS Documentation](#))

Setup complexity

Fargate removes several infrastructure-management tasks associated with running containers on self-managed servers. Users select Fargate as the compute/launch mechanism and configure the ECS or EKS workload.

For ECS, a Fargate task definition specifies resource requirements such as CPU and memory, while AWS manages the underlying compute infrastructure. Fargate tasks can be integrated with VPC networking, load balancers and IAM roles. ([AWS Documentation](#))

The remaining configuration can include:

- ECS/EKS cluster
- Task definitions or Kubernetes manifests
- IAM roles
- VPC/subnets/security groups
- Container images
- Load balancing
- Logging
- Service discovery
- Autoscaling policies

Thus, Fargate abstracts **server provisioning**, but it does not eliminate application, networking, security or orchestration configuration.

Cost structure

Fargate uses a **usage-based pricing model with no upfront charge**. AWS calculates charges according to resources consumed by running tasks/Pods. Current pricing dimensions include:

- vCPU
- Memory
- Operating system
- CPU architecture
- Ephemeral storage

Billing is calculated from when the container image download starts until the ECS task/EKS Pod terminates, with pricing calculated per second and a one-minute minimum. ([Amazon Web Services, Inc.](#))

Fargate provides **20 GB of ephemeral storage by default**; additional configured ephemeral storage is charged separately. Standard AWS data-transfer charges also apply. ([Amazon Web Services, Inc.](#))

For example, AWS's current US East (N. Virginia) pricing example uses Linux/x86 rates of:

- **\$0.000011244 per vCPU-second**
- **\$0.000001235 per GB-memory-second**
- **\$0.0000000308 per GB-second of additional ephemeral storage**

AWS gives a resulting example of **\$1.26/month** for five tasks running for 10 minutes per day, each configured with 1 vCPU, 2 GB memory and 30 GB ephemeral storage. These figures are specifically an AWS pricing example for that region and configuration, not universal Fargate rates. ([Amazon Web Services, Inc.](https://aws.amazon.com/fargate/pricing/))

AWS also documents **Fargate Spot for ECS**, which can provide up to a 70% discount from regular Fargate pricing for interrupt-tolerant tasks; the actual Spot price varies. ([Amazon Web Services, Inc.](https://aws.amazon.com/fargate/pricing/))

Factual reference snapshot

Technology	Core architecture	Documented scalability information	Setup model	Primary cost structure
Kubernetes	Control plane + worker nodes; API server, etcd, scheduler, controllers, kubelet	v1.37 target: 5,000 nodes, 150,000 Pods, 300,000 containers, 110 Pods/node	Self-managed or managed service; production self-management involves multiple infrastructure components	Open-source software; infrastructure, cloud services and operational costs
Docker Swarm	Docker Engine-based managers + workers; Raft-managed cluster state	No equivalent published hard cluster-size target; Docker recommends max 7 managers	Swarm mode is integrated into Docker Engine; initialize manager and join nodes	Open-source Docker Engine/Swarm plus underlying infrastructure; separate Docker commercial-product licensing may apply
AWS Fargate	Managed/serverless container compute for ECS tasks or EKS Pods	Resource and AWS regional quotas; task sizes currently extend to 32 vCPU/244 GB memory	AWS manages underlying compute; customer configures ECS/EKS workloads and AWS resources	Pay-per-use vCPU, memory, OS/architecture and storage; data-transfer and other AWS charges can apply

The industry-adoption figures above are primarily **Kubernetes/CNCF statistics**; the absence of equivalent Swarm or Fargate adoption percentages here reflects the source set rather than an assertion that those technologies have no industry usage. ([CNCF](#))